

Cómo se hizo



**UNIVERSIDAD
DE GRANADA**

Autor: Lucas Hidalgo Herrera

Grado: Doble Grado en Ingeniería Informática y Matemáticas

Asignatura: Programación Web

Profesor: Serafín Moral García

Fecha: 28 de mayo de 2026

Índice General

1	Introducción	2
2	Funciones útiles	2
2.1	PHP	2
3	Conexión a la base de datos	2
4	El paradigma en JavaScript	3
4.1	Tratamiento de asincronía	3
4.2	Las API's	4
5	Gestión de usuarios	5
5.1	Back-end	5
5.1.1	Alta de usuarios dinámica	6
5.2	Front-end	7
6	Gestión de sesiones	9
6.1	Back-end	9
6.2	Front-end	10
7	El administrador	10
8	Buscador de viajes	10
9	Validación de formularios	10
10	Carrusel de viajes	10
11	Bibliografía	10

1 Introducción

A lo largo de este documento, se detallará el proceso de dinamización de la página web de *Azimuth Viajes*. Se tratará en detalle cada paso a seguir; así como los aspectos novedosos que se vayan introduciendo en el proceso de realización.

Antes de continuar debo aclarar que he decidido trabajar con el paradigma de programación dirigida a objetos para *PHP*; de manera que, siempre que se pueda se trabajará con un objeto.

2 Funciones útiles

2.1 PHP

Como cabe esperar, no todas las funciones del back-end que se necesitan pueden estar enmarcadas en una clase que contenga completa funcionalidad; por tanto, he creado un archivo *utils.php* que se encarga de albergar estas funcionalidades; sobre todo, contienen funciones para evitar repetir código o para gestionar el *html* según el tipo de usuario que usa la página web. Estas son:

- *putAvatar*: Es la encargada de escribir en la cabecera el nombre de usuario, su rol y el botón de *log out*; es decir, si es administrador aparece un ordenador y si es usuario aparece el típico avatar indeterminado.
- *putSignUpForm*: Es la encargada de escribir completamente el formulario de inicio de sesión en caso de que ésta no esté iniciada, simplemente es limpieza en el código.
- *putFormSugerencias*: Esta función tiene una funcionalidad análoga a la anterior, su utilidad es limpiar el código del archivo *sugerencias.php* y simplemente escribe el formulario de sugerencias en caso de que la sesión esté iniciada.

3 Conexión a la base de datos

En esta sección, continuamos con la gestión del back-end, es decir, tratando estrictamente con *PHP*; en este caso, hablamos de la conexión a la base de datos. En un inicio, traté de realizarlo con una clase que fuera *Conexion* y se encargará de crear y cerrar la conexión a la base de datos mediante un objeto.

Sin embargo, gracias a lo que había en el guión 3 de prácticas y a que, al realizar las demás clases, debía repetir mucho este código, decidí seguir el esquema del guión. En definitiva, he creado una clase abstracta *DataObject* que se encarga de modelar a un objeto genérico de la base de datos y del cual heredan todos.

Si nos fijamos no he hablado de las credenciales de la base de datos pero sin ellas no podemos conectarnos. Para ello, he seguido las directrices del guión 3 de prácticas que para crear el archivo *config.php* donde se "escondan" las credenciales de la base de datos y todo lo relativo a la conexión. Además, dispone de nombres genéricos para tratar de forma más simple e inequívoca las tablas de la base de datos.

4 El paradigma en JavaScript

Esta sección trata de cómo he gestionado la "comunicación" entre el *front-end* y el *back-end* y es una de las novedades o innovaciones más importantes que he implementado.

Esta se trata del paradigma **AJAJ**, evitando explicaciones es tratar la comunicación como **AJAX** pero con formato *json*. Para ello, me he basado en la gestión de la comunicación asíncrona con *API REST* donde el código *javascript* lanza peticiones *fetch* hacia una *API* para comunicarse con el *back-end*.

4.1 Tratamiento de asincronía

Tal y como he comentado en la sección anterior, estamos ante un paradigma de refresco de página sin recarga y donde *JavaScript* es el protagonista. Al igual que en cualquier comunicación, debemos mandar una pregunta y obtener una respuesta.

En el caso de una comunicación síncrona, aquel que pregunta debe esperar atentamente a que el receptor piense y de una respuesta. Sin embargo, nosotros queremos seguir trabajando mientras el receptor (*back-end*) piensa y formula su respuesta.

Para conseguir eso usaremos las siguientes palabras clave o sentencias:

- *async*: Se usa en la declaración de funciones o métodos de una clase y sirve para determinar que es un método asíncrono. Concretamente, son funciones que devuelven "promesas"; de manera que, si la función devuelve un valor, éste se envuelve en una promesa resuelta y si da error, la promesa será rechazada.

Para entenderlo más fácilmente es como si estoy hablando con una persona, le hago una pregunta y me promete que me va a responder; yo estoy trabajando en otra cosa pero tengo en mente que me tiene que dar una respuesta. Entonces, si esta respuesta es válida y correcta, la procesaré y si no lo es gestionaré el error de comunicación.

- *await*: Sólo puede usarse en entornos gestionados por *async* y se encarga de crear la promesa. Tras esta palabra suele enviarse la pregunta al *back-end* en forma de datos o suele ponerse una función para recoger los datos de respuesta.
- *fetch*: Sólo voy a explicar lo que he usado; esta sentencia es la encargada de mandar la promesa a la *API* que la procesará ¹. La sintaxis básica que yo he usado se compone de los siguientes objetos:

- *endpoint*: Dirección de la *API* a la que mandar los datos o la petición.
- *configurationObject*: Es un objeto de tipo diccionario entre llaves que se encarga de definir cómo va a ser la comunicación; algunos de sus

¹En nuestro caso la *API* será normalmente un código *php* que procesa los datos y devuelve errores o éxitos en formato *json*. Habitualmente, estarán ubicadas en la carpeta `'/php/api'`.

atributos son *method* (método *HTTP* para enviar la petición), *Content-Type* (tipo de contenido que se envía, habitualmente "application/json") y *body* (cuerpo de la petición con los datos en formato *json*).

Aunque esto puede quedar abstracto, veremos un ejemplo de uso en el alta de usuarios. Por último, y a modo de resumen, podemos ubicar que la metodología para trabajar con *fetch* es la siguiente:

1. Definir el método *async*.
2. Creamos la promesa donde nos comunicamos con la *API* a través de *fetch*.
3. Comprobamos si hay respuesta de la *API* y la conexión ha ido como debería². En caso contrario gestionamos el error³.
4. "Esperamos" a recibir la respuesta al mensaje completo en formato *json*.
5. Procesamos los datos ya habiendo finalizado la consulta asíncrona.

En definitiva, es un proceso más o menos sencillo igual que otro. En clase vimos cómo hacerlo con *xmlHttpRequest*; sin embargo, buscando información encontré que este método es más novedoso y se está utilizando actualmente en el desarrollo web. Desconozco que el paradigma visto en clase se siga o no utilizando pero, es cierto, que me pareció más complicado. Por eso, elegí este paradigma; además, es una nueva innovación.

4.2 Las API's

Habitualmente, estas *API's* ya están creadas y con plena funcionalidad para ser usadas por los programadores de una forma específica. Sin embargo, yo debo crearlas. Tal y como comenté en un pie de página en la sección 4.1 están ubicadas en la carpeta 'php/api/' pues no son más que ficheros que se encargan de procesar una entrada en formato *json* y devolver una salida, de nuevo, en formato *json*.

En estas *API's* se debe crear un mensaje *HTTP* al completo con la información a enviar. Las partes que se tratan son:

- *Header*: con la sentencia *Header()* podemos crear y formatear la cabecera del mensaje *HTTP* que vamos a enviar como respuesta.
- *Obtención de datos*: aquí procesamos los datos que debemos recibir; en nuestro caso, usamos la siguiente sentencia:

Donde, debido a que *PHP* no puede leer los datos del *body* accediendo a la variable *\$_POST*, accedemos al flujo de entrada en crudo ("php://input"). Además, 'json_decode' se encarga de decodificar el formato y pasarlo a un *array* asociativo de *PHP*.

²Lo que realmente estamos procesando es la respuesta de la *API* al recibir la cabecera de nuestro mensaje.

³Habitualmente lanzando un error.

- *Procesamiento de datos*: en esta parte, procesamos los datos, cada *API* lo hará a su manera.
- *Enviamos respuesta*: fase en la cual debemos responder a la petición. Habitualmente, se utiliza la siguiente sentencia:

La sentencia *echo* es la forma nativa de escribir datos en el cuerpo de una respuesta *HTTP*; por tanto, estamos enviando la respuesta tras cerrar la ejecución del *script* y codificando en modo *json* el *array* asociativo con las respuestas.

Por tanto, ya hemos visto todo lo necesario para trabajar con el *front-end* y *back-end* de forma asíncrona con *fetch*.

5 Gestión de usuarios

Entramos ya en aspectos importantes, una aplicación web no es nada sin usuarios; por tanto, debemos crear una tabla para ellos en la base de datos. Dicha tabla ('Users') se encarga de modelar a un usuario que dispondrá de los siguientes atributos:

- Nombre
- NickName (PK)
- Email (Unique)
- Password: Este atributo siempre debe aparecer cifrado para que no sea visible para ninguna persona con intenciones maliciosas.
- Admin: Este atributo refleja si el usuario es administrador o no, por defecto, tomará el valor 0 pues el único administrador ya está creado en la base de datos y es el usuario cuyo nickname es 'el_dramas1s'.

Con esta representación podremos gestionar a los usuarios de forma más o menos cómoda.

5.1 Back-end

Con respecto al cómo se ha modelado para el trato en el servidor (*PHP*), he creado una nueva clase en el archivo *user.php* que hereda de *DataObject* y que modela exactamente las características de los usuarios.

Esta clase, que al principio modelé como una clase instanciable y otra clase distinta que lo manejaba, se encarga ahora de hacer ambas cosas usando métodos de clase y métodos de objeto.

Referente a los métodos de objetos, son los encargados de gestionar las operaciones de la base de datos, así como de realizar algunas verificaciones de los objetos en

cuestión. Estos métodos crean usuarios en la base de datos, los eliminan, los buscan y verifican que la contraseña y el *nickName* pasados como argumentos estén asociados.

Para el caso de los métodos de objeto, tenemos un método *getter* para obtener los datos, un método de validación de consistencia de un usuario y un método para comprobar si es admin.

5.1.1 Alta de usuarios dinámica

Comienza lo interesante, debemos ahora gestionar la alta de nuevos usuarios en la página web; por tanto, deberemos tener en cuenta que debemos comprobar el formulario de alta de usuario. Esta tarea no solo es del front-end sino que también del back-end pues nos dará mayor protección frente a fallos del usuario o del front-end.

Por tanto, he creado un archivo *forms.php* que se encarga de manejar los formularios. Este archivo sigue la misma lógica de árbol de herencia que los *DataObjects* pues disponemos de un manejador que se encarga de modelar el comportamiento básico como una clase abstracta que aparece en el *listing ??*.

Donde aparecen las cuatro funciones básicas de un formulario:

- **Validar entradas:** La función *validate* se encarga de validar las entradas del objeto, cada formulario tendrá sus validaciones dependiendo del objeto que se esté tratando.
- **Procesar entradas:** La función *process* se encarga de procesar el formulario dándole la funcionalidad que merece. Dependiendo del formulario que se trate realizará unas operaciones u otras.
- **Gestionar posibles errores:** Esta funcionalidad la realizan dos funciones, *addError* y *getErrors*, junto con el dato miembro de la clase que es el array de errores.

Por tanto, cada vez que queramos gestionar un formulario, lo haremos a través de la función *handle* que engloba las dos primeras. Y si devuelve *false* procesaremos los errores con *getErrors*.

Nos centramos ya en el formulario de alta de usuarios, cuyo manejador es *Form-SignUp* que se encarga de obtener los datos, validarlos y, si cumple las características básicas para poder crear un usuario, se crea en la base de datos. La única característica que queda fuera de las básicas es una condición de política; esta es: "un usuario no puede ser menor de edad"⁴ ⁵.

El código de este manejador aparece en el *listing ??*.

⁴El motivo de esto es evitar pagos de usuarios menores de edad, en caso de falsificarlo es problema del usuario.

⁵Si nos fijamos, no había ningún atributo de mayoría de edad en la base de datos para el usuario; el motivo es que si está en la base de datos es porque se mayor de edad.

Ahora, pensando en como funciona nuestra comunicación con el *front-end* a través de *fetch*, debemos crear una *API*; tal y como comenté en la sección ??, se encuentra en el archivo *altausuarios.php* dentro de la carpeta */php/api*. Este archivo contiene el siguiente código:

En él aparece la gestión, básicamente procesa entradas desde *javascript* hacia el *back-end* creando un manejador de formularios y aplicando el método *handle*. De esta forma, el archivo *altausuarios.html* no cambia. Además, con la gestión realizada con *javascript* que realizaré después no es necesario utilizar el archivo *validacion.html* que teníamos en la segunda práctica. Por tanto, esta *API* procesa los datos recibidos en formato *json* y devuelve una respuesta en cuyo *body* está en formato *json*.

5.2 Front-end

Para entender esto, lo primero que debemos entender es **¿qué debe hacer el *front-end* en esta parte?**. Concretamente, debe comprobar que los datos del formulario de alta de usuarios sean correctos. Para ello, vamos a usar la clase *formularioAlta* que hereda de *formularioBase* y de la cual heredarán todos los formularios.

Comencemos explicando un poco el código de *formularioBase*; la idea es clara, emular el modelo realizado en *php* sobre los formularios. Por tanto, esta clase se tomará como clase abstracta que define el comportamiento completo de un formulario cualquiera. Además, como no tenemos que conectar con la base de datos al estar en *front-end* es más fácil de definir el comportamiento. Este comportamiento se basa en un *eventListener* que se activa al darle al botón de enviar y funciona de la siguiente forma:

1. Anulamos el comportamiento por defecto del navegador ante este evento con *preventDefault* para evitar que recargue la página, queremos controlar todo lo que pasa en el navegador.
2. Obtenemos las reglas del formulario que hemos instanciado. Las reglas es una función que dependerá del formulario en cuestión y las definimos como una función encargada de devolver funciones según el valor del campo que estamos comprobando; se puede entender como una receta de recetas. En estas reglas podemos definir el comportamiento que ve el usuario, como por ejemplo, hacer "click" en un enlace obligatorio.
3. Limpiamos los errores que hubiera en un principio en caso de que no fuera el primer intento del formulario para poder poner otros en caso de corregir los que había. La función de limpieza es clara; puesto que lo que hago para mostrar los errores es crear un contenedor con el error dentro de un contenedor que contiene los campos, bastará con eliminarlo completamente para que, en caso

de necesitarlo, crear otro nuevo y no modificar demasiado el espaciado entre campos ⁶.

4. Obtenemos los datos del formulario con *FormData*⁷ y los convertimos en una array asociativo con la función *Object.fromEntries()*.
5. Validamos los datos dle formulario aplicando las reglas a cada uno de ellos.
6. Si hay errores los mostramos creando un contenedor de error dentro del contenedor del campo. En el caso de tener un *radioButton* lo que hacemos es tomar el primero de ellos como referente para la construcción del error.
7. Si no hay errores realizamos la comunicación con *fetch* usando el *endpoint* del formulario. Cabe destacar que cada uno dispone de su *endpoint* o *API* del *back-end* para procesar sus envíos.

Además, si el manejador de formulario del *back-end* detecta algún error como ser mayor de edad, lo devuelve en el *body* del mensaje *HTTP* de respuesta y al procesarla en el *front-end* se muestra el error en el campo correspondiente.

Es importante recalcar que la funcion de envío se trata de forma asíncrona como se explicó en la sección 4.1. Además, hay una cosa que no he comentado y que es un añadido, pues con lo que he explicado todo funciona correctamente. Si miramos el código, en el constructor aparece la función *realTimeValidation* que, como dice su nombre, se encarga de procesar la validación en tiempo real del formulario.

Esta funcion es sencilla, simplemente hay que añadir un *eventListener* al formulario por cada evento del tipo que queramos aplicando las reglas y limpiando los errores si todo está correcto. En caso de no estar correcto, simplemente muestra el error.

Pasando ahora al comportamiento concreto del alta de usuarios, éste se define en el archivo *formularioAlta.js* donde está la clase *formularioAlta* y que hereda de *formularioBase*. Por tanto, solo debemos asignar el *endpoint* que en este caso es *../php/api/altausuarios.php*, definir el booleano de *avisoLeido* para forzar al usuario a leer el aviso legal simulando términos y condiciones y crear las reglas del formulario.

Antes de explicar las reglas, me gustaría explicar una experiencia que tuve con *avisoLeido*. En un inicio, esta bandera se gestionaba en la función de reglas, es decir, se ponía a falso y, con un *eventListener* en el enlace se cambiaba a *true* cuando se hacía "click". Sin embargo, tras un largo tiempo pensando por qué no funcionaba, me di cuenta de que no era una gestión correcta pues, cada vez que se ejecutaban las reglas, notaba como que el enlace no era "clickado" lo cual era cierto. Por tanto, decidí pasarlo a dato miembro del objeto, lo cual permitía guardar el cambio de form apermamente.

⁶Esto hace referencia a que, una vez probé a crear los contenedores de error y simplemente cambiar el interior a "" si no existía ese error. Esto solo consiguió que aumentara el gap entre campos de forma notable.

⁷Es una interfaz nativa que permite construir fácilmente conjuntos de datos en pares clave-valor para enviarlos al servidor

6 Gestión de sesiones

Pasamos a algo importante, cada usuario debe tener una sesión y para ello, debemos usar el formulario de inicio de sesión. Una vez el usuario ya tiene un perfil creado, puede loguearse y ver aquellas funcionalidades más importantes.

6.1 Back-end

En el servidor seguiremos la misma lógica que con la creación de usuarios; debemos gestionar el formulario de inicio de sesión. Utilizaremos una *API* que procese los datos recibidos por *fetch* y actuaremos arreglo a esos usando un manejador del formulario. En este caso es muy sencillo, simplemente comprueba que las credenciales coinciden en la base de datos aplicando el *hash* correspondiente y buscando si el usuario existe y, si todo es correcto almacena información del usuario en el array `$_SESSION`. El código de esta parte es el siguiente:

Una vez ya hemos hablado del formulario y de la *API*, que son bastante sencillos, debemos de hablar sobre cómo utiliza esto el servidor para mostrar unas cosas u otras. Trataremos el código de *index.php* y con eso entenderemos el funcionamiento de otros archivos que usen estos aspectos.

Como vemos, siempre que queramos hacer uso de la sesión debemos iniciarla con la sentencia *session_start()*. Realmente, no siempre la iniciamos, el funcionamiento que he encontrado es el siguiente:

- **PHPSESSID**: Cuando iniciamos una sesión por primera vez, el servidor envía una *cookie* que contiene el id de sesión y crea un archivo con la información del array `$_SESSION`.
- *session_start()*: Cuando usamos esta función en *php* lo que hacemos es comprobar que el navegador ha mandado ese id de sesión al servidor, en caso de que lo tengamos ⁸. El servidor reinicia la conversación y podemos usar la información almacenada. Además, se encarga de aumentar el tiempo de vida de la sesión.

Los aspectos que tratamos en algunos archivos son los siguientes:

- **Perfil del usuario**: Con esto me refiero a la imagen de perfil y el *nickName*. Cuando ya se ha iniciado sesión, se recarga la página y el formulario de inicio de sesión se cambia por el nombre de usuario junto a su rol y el botón de cierre de sesión.
- **Sugerencias**: Para la empresa *Azimut Viajes*, una sugerencia sin autor no es nada; por tanto, debes ser usuario para poder dejar una sugerencia.

⁸Si no lo tenemos hacemos el punto anterior.

- **Buscador:** Considero que es buena idea que, si el usuario no se ha registrado, no pueda ser capaz de realizar búsquedas. En caso de que quiera ver los viajes tendrá la posibilidad de verlos todos en la sección de viajes de la página web.

Por último, aunque lo explique en la sección 7, cabe recalcar que para gestionar las funcionalidades del administrador se hace uso del campo *admin* del array asociativo de las sesiones. Dependiendo de su valor, rederizamos unas cosas u otras ⁹.

6.2 Front-end

Con lo explicado en la sección 5 hay poco que explicar. Solo debemos saber que para gestionar esto debemos recaer en una nueva clase hija de *formularioBase*; ésta se llamara *formularioSesion* y se encargará de tomar el *endpoint* correcto y de gestionar las reglas de validación oportunas. El código es el siguiente:

Como podemos ver, el diseño que he realizado para procesar elementos en el *front-end* es bastante cómodo y limpio. ¡Se me olvidaba! Debemos añadir el *script* al archivo *index.php* y, también, añadir una nueva sentencia al archivo *main.js*; aparece justo aquí debajo esa sentencia:

7 El administrador

8 Buscador de viajes

9 Validación de formularios

10 Carrusel de viajes

11 Bibliografía

⁹Considero que esa es la mejor opción, pues mandarlas como invisibles permite que el navegador conozca los aspectos que trata un administrador y hacerlos visibles es tan fácil como modificar el código en el navegador.